Transformers, Pretraining and Fine-Tuning

Prepared by Amritanshu Singh

1 Introduction

Transformers are an important development in deep learning and natural language processing. They are used in modern language models for understanding text, generating responses, translation, summarization, and many other applications.

The main idea behind Transformers is attention. It allows a model to identify relationships between different parts of a sequence instead of processing every word independently.

Modern language models are generally developed through two important stages: pretraining and fine-tuning. Pretraining helps the model learn general language patterns, while fine-tuning adapts the model to a particular task.

2 Transformer Architecture

A Transformer is a neural network architecture that processes text by converting it into numerical representations and learning relationships between tokens.

Its main components are tokenization, embeddings, positional information, attention, and feed-forward neural networks.

2.1 Tokenization and Embeddings

Tokenization converts text into smaller units called tokens. These tokens may represent complete words, parts of words, or other text units.

The model converts each token into a numerical vector called an embedding. These vectors allow the neural network to process textual information mathematically.

2.2 Positional Information

The order of words is important for understanding a sentence. Positional information helps the Transformer distinguish between tokens at different positions.

It is combined with token representations so that the model can learn from both the content and the order of the input.

2.3 Attention Mechanism

Attention is the central component of a Transformer. It allows each token to gather information from other relevant tokens in the sequence.

For example, the meaning of a word can depend on the words around it. Attention helps the model represent these relationships and understand context.

2.4 Multi-Head Attention

Multi-head attention uses several attention operations in parallel. Different heads can learn different relationships between tokens.

Their outputs are combined to produce richer representations of the input text.

2.5 Feed-Forward Networks

Feed-forward networks further process the representations created by attention. Residual connections and normalization help information pass through the network during training.

These components are repeated in multiple Transformer layers to build more complex representations.

3 Transformer Architecture Diagram

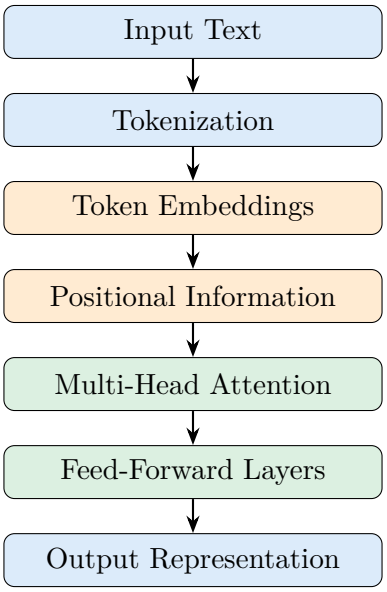

Figure 1: Basic Transformer architecture

4 Types of Transformer Models

4.1 Encoder-Only Models

Encoder-only models are mainly designed to understand and represent input text. They are useful for text classification, sentiment analysis, and information extraction.

BERT is a common example of an encoder-only model.

4.2 Decoder-Only Models

Decoder-only models generate text by predicting the next token based on previously available tokens.

They are commonly used in conversational systems, text generation, and code generation. GPT-style models belong to this category.

4.3 Encoder-Decoder Models

Encoder-decoder models contain both an encoder and a decoder. The encoder processes the input, while the decoder generates the output.

These models are useful for translation, summarization, and other tasks involving input and output sequences.

5 Pretraining

Pretraining is the process of training a model on a large collection of text before adapting it to a specific task.

During this stage, the model learns general language patterns, relationships between words, sentence structure, and contextual information. The training process repeatedly compares model predictions with training targets and adjusts the parameters to improve its predictions.

5.1 Pretraining Process

The main stages of pretraining are:

- Collecting a large text dataset.
- Cleaning and preparing the text.
- Converting the text into tokens.
- Training the model using a language-learning objective.
- Saving the trained model for later use.

5.2 Autoregressive Pretraining

Autoregressive models learn to predict the next token using the tokens that appear before it.

For example, when given the beginning of a sentence, the model learns to predict likely continuations based on patterns learned during training.

This approach is commonly used in decoder-only language models and helps them generate text one token at a time.

5.3 Masked Language Modeling

Masked language modeling trains a model to predict selected tokens that have been hidden from the input.

For example:

> The student is reading a [MASK].

The model uses the surrounding context to predict the missing token. BERT uses this approach during pretraining.

5.4 Importance of Pretraining

Pretraining gives a model a general foundation before it is applied to a particular task. Training a large model from scratch requires substantial data and computing resources.

A pretrained model can be reused and adapted for different applications, making development more practical.

6 Fine-Tuning

Fine-tuning is the process of adapting a pretrained model to a specific task using a more focused dataset.

Instead of learning everything from the beginning, the model starts with the representations learned during pretraining and adjusts them to suit the target application.

For example, a pretrained language model can be adapted to classify customer feedback, summarize documents, or answer questions within a particular domain.

6.1 Fine-Tuning Process

The main stages of fine-tuning are:

- Selecting a suitable pretrained model.
- Preparing data for the target task.
- Training the model on task-specific examples.
- Saving the adapted model for use.

6.2 Supervised Fine-Tuning

Supervised fine-tuning uses examples containing inputs and their desired outputs.

The model learns to produce predictions that match these outputs. It is commonly used for classification, instruction following, question answering, and summarization.

6.3 Full Fine-Tuning

In full fine-tuning, most or all of the model parameters are updated using the new dataset.

This provides flexibility but may require significant computational resources and memory.

6.4 Parameter-Efficient Fine-Tuning

Parameter-efficient fine-tuning updates only a small part of the model or introduces a limited number of trainable parameters.

LoRA (Low-Rank Adaptation) is a popular example. It reduces the number of parameters that need to be trained and can make adaptation more practical when computing resources are limited.

7 Fine-Tuning Architecture

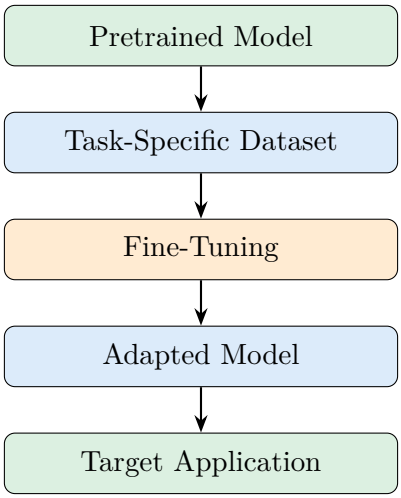

Figure 2: Fine-tuning workflow

8 Pretraining and Fine-Tuning

Pretraining and fine-tuning serve different purposes but work together in the development of language models.

Pretraining uses a large and diverse dataset to learn general language patterns. Fine-tuning uses a more focused dataset to adapt those capabilities to a particular task.

For example, a pretrained model may already be capable of processing natural language. Fine-tuning can adapt it to recognize sentiment in reviews or follow a specific instruction format.

This approach reduces the need to train a new model from scratch for every application.

9 Conclusion

Transformers have become a major part of modern artificial intelligence because attention allows them to learn relationships between tokens and use contextual information.

Pretraining develops general language capabilities using large text collections. Fine-tuning then adapts a pretrained model to a specific task using focused training data.

Together, these techniques provide a practical foundation for building language models for text understanding, generation, and other natural language processing tasks.